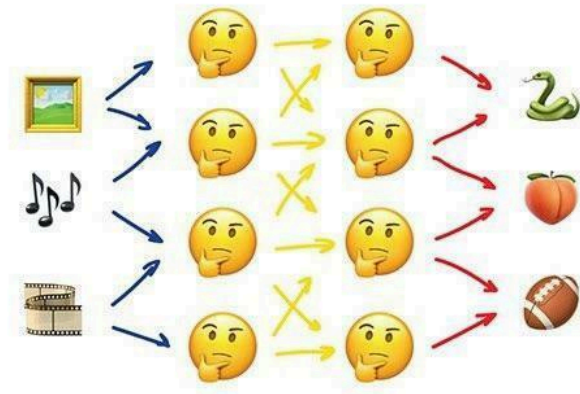


LAPORAN TUGAS BESAR 2

IF3270 PEMBELAJARAN MESIN

“CONVOLUTIONAL NEURAL NETWORK (CNN) & RECURRENT NEURAL NETWORK (RNN)”



Neural Networks

Oleh Kelompok “HidupSpinwheel”:

13523125 Dita Maheswari

13523127 Boye Mangaratua Ginting

13523138 Samantha Laqueenna Ginting

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2026**

DAFTAR ISI

DAFTAR ISI.....	2
BAB I.	
DESKRIPSI PERSOALAN.....	3
BAB II.	
PEMBAHASAN.....	4
2.1. Penjelasan Implementasi.....	4
2.1.1. Deskripsi Kelas serta Atribut dan Method.....	4
2.1.2. Penjelasan Forward Propagation.....	5
2.1.3. Penjelasan Backward Propagation dan Weight Update.....	5
2.2. Hasil Pengujian.....	5
2.2.1. Pengaruh Depth dan Width.....	5
2.2.2. Pengaruh Fungsi Aktivasi.....	5
BAB III.	
KESIMPULAN DAN SARAN.....	6
3.1. Kesimpulan.....	6
3.2. Saran.....	6
REFERENSI.....	7
LAMPIRAN.....	8
A. Pembagian Tugas Anggota.....	8
B. Github Repository.....	8
C. Form Penggunaan AI.....	8

BAB I.

DESKRIPSI PERSOALAN

Tugas besar ini berfokus pada pengembangan dan pemahaman mendalam mengenai Convolutional Neural Network (CNN), Simple Recurrent Neural Network (RNN), dan Long Short-Term Memory (LSTM), yang merupakan pilar utama dalam pemrosesan data spasial dan sekuensial. Karakteristik utama dari CNN adalah kemampuannya dalam mengekstraksi fitur spasial melalui operasi konvolusi, sementara RNN dan LSTM dirancang untuk menangani data berurutan dengan mempertahankan informasi melalui hidden state. Pada tugas besar ini, kami harus mengimplementasikan modul forward propagation untuk ketiga arsitektur tersebut from scratch.

Modul yang dikembangkan dirancang secara modular agar setiap layer memiliki fungsionalitas mandiri yang menerima dan mengembalikan numpy array. Untuk bagian CNN, komponen yang diimplementasikan mencakup layer Conv2D (baik dengan shared maupun non-shared parameter), berbagai jenis Pooling, serta Flatten. Sedangkan untuk bagian RNN dan LSTM, implementasi mencakup Embedding layer, RNN cell, LSTM cell, serta Dense projection. Model yang dibuat harus mampu memuat bobot (load weights) yang telah dilatih menggunakan pustaka Keras untuk kemudian dijalankan dalam mekanisme inference buatan sendiri.

Dari sisi fungsionalitas, tugas ini mencakup integrasi arsitektur encoder-decoder untuk menyelesaikan permasalahan Image Captioning. Fitur utama dalam pipeline ini adalah penggunaan CNN sebagai encoder untuk mengekstraksi vektor fitur dari gambar, yang kemudian diinjeksikan ke dalam RNN atau LSTM sebagai decoder untuk menghasilkan deskripsi tekstual. Proses pembangkitan teks dilakukan dengan metode teacher forcing selama pelatihan dan pengujian efektivitasnya dilakukan melalui perbandingan berbagai variasi arsitektur.

Untuk menguji keandalan implementasi, model akan dilatih dan dievaluasi menggunakan dataset Intel Image Classification untuk tugas visi komputer dan dataset Flickr8k untuk tugas image captioning. Pengujian mencakup analisis pengaruh hyperparameter seperti jumlah layer konvolusi, ukuran filter, jumlah filter, jenis pooling, hingga ukuran hidden state pada model sekuensial. Sebagai tahap akhir, hasil prediksi dari model buatan sendiri akan dibandingkan dengan pustaka standar Keras menggunakan metrik macro F1-score, BLEU-4, dan waktu eksekusi untuk memvalidasi efektivitas serta efisiensi model yang telah dikembangkan.

BAB II. PEMBAHASAN

2.1. Penjelasan Implementasi

2.1.1. Deskripsi Kelas serta Atribut dan *Method*

Berikut adalah penjelasan setiap kelas serta atribut dan *method*-nya:

Class Layer (Base Class)	
<pre>def load_weights_from_keras(self, keras_layer)</pre>	Method abstrak untuk memuat bobot dari layer Keras; harus diimplementasikan oleh subkelas
<pre>def forward(self, x: np.ndarray) -> np.ndarray</pre>	Method abstrak untuk melakukan forward pass; harus diimplementasikan oleh subkelas
<pre>def __call__(self, x: np.ndarray) -> np.ndarray</pre>	Memungkinkan instance kelas dipanggil seperti fungsi; memanggil forward(x)
Class Conv2D (Inherits Layer)	
<pre>filters: int kernel_size: tuple strides: tuple padding: str activation_fn: callable kernel: np.ndarray bias: np.ndarray</pre>	Atribut konfigurasi jumlah filter, ukuran kernel, stride, padding, fungsi aktivasi, serta bobot kernel dan bias
<pre>def load_weights_from_keras(self, keras_layer)</pre>	Memuat bobot kernel dan bias dari layer Conv2D Keras
<pre>def _pad(self, x: np.ndarray) -> np.ndarray</pre>	Melakukan zero-padding pada input sesuai konfigurasi padding ("same" atau "valid")
<pre>def forward(self, x: np.ndarray) -> np.ndarray</pre>	Melakukan operasi konvolusi 2D pada input x menggunakan kernel bersama (shared weights) di setiap posisi spasial
Class LocallyConnected2D (Inherits Layer)	

<pre> filters: int kernel_size: tuple strides: tuple padding: str activation_fn: callable kernel: np.ndarray bias: np.ndarray H_out: int W_out: int </pre>	Atribut konfigurasi filter, kernel, stride, padding, fungsi aktivasi, bobot, bias, serta dimensi output H dan W
<pre> def load_weights_from_keras(self, keras_layer) </pre>	Memuat bobot kernel dan bias dari layer LocallyConnected2D Keras
<pre> def forward(self, x: np.ndarray) -> np.ndarray </pre>	Melakukan konvolusi lokal di mana setiap posisi spasial menggunakan set bobot yang berbeda (tidak berbagi bobot)
Class MaxPooling2D (Inherits Layer)	
<pre> pool_size: tuple strides: tuple </pre>	Atribut ukuran pooling window dan stride
<pre> def load_weights_from_keras(self, keras_layer) </pre>	Memuat konfigurasi pool_size dan strides dari layer MaxPooling2D Keras
<pre> def forward(self, x: np.ndarray) -> np.ndarray </pre>	Mengambil nilai maksimum dari setiap region pooling pada input x
Class AveragePooling2D (Inherits Layer)	
<pre> pool_size: tuple strides: tuple </pre>	Atribut ukuran pooling window dan stride
<pre> def load_weights_from_keras(self, keras_layer) </pre>	Memuat konfigurasi pool_size dan strides dari layer AveragePooling2D Keras
<pre> def forward(self, x: np.ndarray) -> np.ndarray </pre>	Menghitung nilai rata-rata dari setiap region pooling pada input x
Class Flatten (Inherits Layer)	
<pre> def load_weights_from_keras(self, keras_layer) </pre>	Tidak melakukan apapun karena Flatten tidak memiliki bobot yang perlu dimuat
<pre> def forward(self, x: np.ndarray) -> np.ndarray </pre>	Meratakan (flatten) tensor input multidimensi menjadi vektor 1D

	dengan mempertahankan dimensi batch
Class Dense (Inherits Layer)	
<pre>units: int activation_fn: callable W: np.ndarray b: np.ndarray</pre>	Atribut jumlah neuron output, fungsi aktivasi, matriks bobot W, dan vektor bias b
<pre>def load_weights_from_keras(self, keras_layer)</pre>	Memuat bobot W dan bias b dari layer Dense Keras
<pre>def forward(self, x: np.ndarray) -> np.ndarray</pre>	Menghitung transformasi linear lalu menerapkan fungsi aktivasi pada hasilnya
Class Embedding (Inherits Layer)	
<pre>W: np.ndarray</pre>	Matriks embedding berukuran (vocab_size, embed_dim)
<pre>def load_weights_from_keras(self, keras_layer)</pre>	Memuat matriks embedding W dari layer Embedding Keras
<pre>def forward(self, x: np.ndarray) -> np.ndarray</pre>	Melakukan pencarian embedding: mengembalikan baris ke-x dari matriks W untuk setiap token dalam input
Class SimpleRNNCell (Inherits Layer)	
<pre>units: int W_i: np.ndarray W_h: np.ndarray b: np.ndarray</pre>	Atribut jumlah unit hidden, bobot input W_i , bobot hidden W_h , dan bias b
<pre>def load_weights_from_keras(self, keras_layer)</pre>	Memuat bobot W_i , W_h , dan bias b dari layer SimpleRNN Keras
<pre>def forward(self, x_t: np.ndarray, h_prev: np.ndarray) -> np.ndarray</pre>	Menghitung hidden state baru
Class LSTMCell (Inherits Layer)	
<pre>units: int W_i: np.ndarray W_h: np.ndarray b: np.ndarray</pre>	Atribut jumlah unit hidden, bobot input W_i , bobot hidden W_h , dan bias b (mencakup 4 gate sekaligus)

<pre>def load_weights_from_keras(self, keras_layer)</pre>	Memuat bobot W_i , W_h , dan bias b dari layer LSTM Keras
<pre>def forward(self, x_t: np.ndarray, h_prev: np.ndarray, c_prev: np.ndarray)</pre>	Menghitung hidden state h_{next} dan cell state c_{next} menggunakan empat gate: input gate (i), forget gate (f), output gate (o), dan gate g. Mengembalikan tuple (h_{next}, c_{next})

2.1.2. Penjelasan *Forward Propagation* CNN

Proses *forward propagation* pada arsitektur *Convolutional Neural Network* (CNN) yang diimplementasikan bertujuan untuk mengekstraksi fitur spasial dari data masukan berupa citra. Tahapan ini melibatkan beberapa operasi utama, yaitu konvolusi, aktivasi, dan *pooling*. Berikut adalah rincian teknis dari setiap tahapannya:

1. Operasi Konvolusi (Convolution Layer)

Operasi konvolusi adalah inti dari ekstraksi fitur. Pada implementasi kelas *Conv2D*, *forward propagation* dilakukan dengan menerima masukan berupa tensor berdimensi empat (N, H, W, C), di mana N adalah jumlah *batch*, H adalah tinggi, W adalah lebar, dan C adalah jumlah saluran (*channels*).

Proses matematis yang terjadi adalah sebagai berikut:

- **Kernel Sliding:** Sebuah *filter* atau *kernel* berukuran (k_h, k_w, C) digeser di atas gambar masukan dengan langkah tertentu (*stride*).
- **Dot Product:** Pada setiap posisi pergeseran, dilakukan perkalian elemen demi elemen antara nilai *pixel* pada jendela masukan dengan bobot pada *kernel*.
- **Summation & Bias:** Hasil perkalian tersebut dijumlahkan menjadi satu nilai skalar, kemudian ditambahkan dengan *bias*. Secara formal, nilai pada koordinat (i, j) di saluran *output* f dihitung dengan rumus:

$$O_{i,j,f} = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} \sum_{c=0}^{C-1} (X_{i+m,j+n,c} \cdot W_{m,n,c,f}) + b_f$$

- Shared Parameters: Sifat utama dari Conv2D adalah penggunaan bobot yang sama (*shared weights*) untuk seluruh area gambar, yang secara drastis mengurangi jumlah parameter dibandingkan lapisan *fully connected*.

Selain itu, terdapat variasi berupa LocallyConnected2D di mana parameter tidak dibagikan (*non-shared parameters*). Pada lapisan ini, setiap lokasi spasial pada gambar masukan memiliki set bobotnya sendiri, yang berguna untuk menangkap fitur yang bersifat spesifik pada lokasi tertentu.

2. Fungsi Aktivasi

Setelah mendapatkan hasil konvolusi, nilai tersebut dilewatkan ke fungsi aktivasi untuk memperkenalkan sifat non-linearitas ke dalam model. Fungsi ReLU (Rectified Linear Unit) digunakan secara standar, yang mengubah semua nilai negatif menjadi nol:

$$f(x) = \max(0, x)$$

Hal ini memungkinkan jaringan untuk mempelajari pola yang lebih kompleks dan mempercepat proses konvergensi saat pelatihan.

3. Operasi Pooling (Downsampling)

Lapisan *pooling* digunakan untuk mengurangi dimensi spasial (*spatial size*) dari *feature map*, sehingga mengurangi beban komputasi dan membantu model menjadi lebih invarian terhadap pergeseran kecil pada gambar. Terdapat dua jenis *pooling* yang diimplementasikan:

- Max Pooling: Mengambil nilai maksimum dari jendela yang ditentukan. Operasi ini efektif untuk mempertahankan fitur yang paling menonjol (seperti tepi atau tekstur tajam).
- Average Pooling: Mengambil nilai rata-rata dari seluruh elemen dalam jendela. Operasi ini memberikan hasil yang lebih halus dibandingkan *max pooling*.

Secara teknis, proses *forward* pada kelas MaxPooling2D dan AveragePooling2D bekerja dengan melakukan iterasi pada setiap

channel secara independen dan menerapkan fungsi `np.max` atau `np.mean` pada setiap jendela geser.

4. Perataan (Flattening)

Sebagai tahap akhir dari bagian *feature learning* sebelum masuk ke *classifier*, data yang berbentuk tensor multi-dimensi diratakan menjadi vektor satu dimensi. Proses ini dilakukan dengan mengubah dimensi (H, W, C) menjadi sebuah vektor berukuran $H \times W \times C$, sehingga dapat diterima oleh lapisan *Fully Connected* (Dense).

2.1.3. Penjelasan *Forward Propagation* RNN

Implementasi *Forward Propagation* pada *Recurrent Neural Network* (RNN) dirancang untuk menangani data yang bersifat sekuensial, di mana urutan informasi sangat menentukan hasil keluaran. Berbeda dengan CNN yang memproses data secara spasial, RNN memiliki mekanisme memori melalui *hidden state* yang diteruskan dari satu langkah waktu ke langkah waktu berikutnya.

Proses *forward propagation* pada unit dasar `SimpleRNNCell` mengikuti tahapan berikut:

1. Input dan Hidden State

Pada setiap langkah waktu t , unit RNN menerima dua masukan utama:

- x_t (Input Current Step): Representasi fitur dari urutan saat ini (misalnya, vektor *embedding* dari sebuah kata atau fitur gambar yang diekstraksi).
- h_{t-1} (Previous Hidden State): Informasi yang dibawa dari langkah waktu sebelumnya.

2. Transformasi Linear dan Integrasi Informasi

Di dalam metode forward, terjadi penggabungan antara input saat ini dengan memori masa lalu. Hal ini dilakukan dengan mengalikan masing-masing masukan dengan matriks bobot yang berbeda:

- W_{xh} : Matriks bobot untuk input terhadap *hidden state*.

- W_{hh} : Matriks bobot untuk *recurrent connection* (antara *hidden state* sebelumnya ke *hidden state* saat ini).

Integrasi ini dihitung dengan rumus:

$$z_t = (x_t \cdot W_{xh}) + (h_{t-1} \cdot W_{hh}) + b_h$$

Di mana b_h adalah vektor *bias* yang ditambahkan untuk meningkatkan fleksibilitas model dalam memetakan data.

3. Aktivasi Non-Linear

Hasil dari integrasi linear z_t kemudian dilewatkan melalui fungsi aktivasi non-linear. Dalam implementasi kelas SimpleRNNCell ini, digunakan fungsi Tanh (Hyperbolic Tangent). Fungsi ini memetakan nilai ke dalam rentang $(-1, 1)$, yang membantu menstabilkan nilai *hidden state* agar tidak meledak (*exploding*) selama proses iterasi sekuens.

$$h_t = \tanh(z_t)$$

4. Mekanisme Iterasi (Unrolling)

Meskipun SimpleRNNCell hanya memproses satu langkah waktu, lapisan RNN secara keseluruhan bekerja dengan melakukan iterasi (pengulangan) sebanyak panjang sekuens (T).

- Pada $t = 0$, *hidden state* awal (h_0) biasanya diinisialisasi sebagai vektor nol.
- *Output* h_t dari langkah pertama menjadi masukan bagi langkah kedua, dan seterusnya hingga $t = T$.
- Hasil akhir dari proses *forward* ini dapat berupa seluruh urutan *hidden states* atau hanya *hidden state* terakhir, tergantung pada kebutuhan arsitektur (misalnya untuk klasifikasi atau *sequence generation*).

Dengan mekanisme ini, RNN dapat mempertahankan informasi dari elemen-elemen sebelumnya dalam sebuah urutan, yang sangat penting untuk tugas-tugas seperti *image captioning* di mana kata yang dihasilkan bergantung pada konteks kata-kata sebelumnya.

2.1.4. Penjelasan *Forward Propagation* LSTM

Arsitektur *Long Short-Term Memory* (LSTM) merupakan pengembangan dari RNN yang dirancang khusus untuk mengatasi masalah *vanishing gradient* pada sekuens yang panjang. Perbedaan utama LSTM dibandingkan RNN standar terletak pada adanya *cell state* (c_t) yang berfungsi sebagai jalur informasi jangka panjang, serta mekanisme *gating* yang mengatur informasi mana yang boleh masuk, keluar, atau dihapus.

Pada implementasi kelas LSTMCell, proses *forward propagation* dilakukan melalui empat tahapan gerbang (*gates*) sebagai berikut:

1. Forget Gate

Gerbang ini menentukan informasi apa dari *cell state* sebelumnya (c_{t-1}) yang akan dibuang. Masukan saat ini (x_t) dan *hidden state* sebelumnya (h_{t-1}) dilewatkan melalui fungsi aktivasi Sigmoid. Nilai yang mendekati 0 berarti informasi tersebut akan dilupakan, sedangkan nilai mendekati 1 berarti informasi dipertahankan.

$$f_t = \sigma(x_t \cdot W_{xf} + h_{t-1} \cdot W_{hf} + b_f)$$

2. Input Gate dan Candidate State

Tahap ini menentukan informasi baru apa yang akan disimpan ke dalam memori. Terdiri dari dua bagian utama:

- Input Gate (i_t): Menggunakan fungsi Sigmoid untuk memutuskan bagian mana dari informasi baru yang akan diperbarui.
- Candidate Cell State ($\tilde{c}_t$): Menggunakan fungsi Tanh untuk menciptakan kandidat nilai baru yang bisa ditambahkan ke dalam *cell state*.

$$i_t = \sigma(x_t \cdot W_{xi} + h_{t-1} \cdot W_{hi} + b_i)$$

$$\tilde{c}_t = \tanh(x_t \cdot W_{xc} + h_{t-1} \cdot W_{hc} + b_c)$$

3. Cell State Update

Setelah mendapatkan sinyal dari *forget gate* dan *input gate*, *cell state* saat ini (c_t) diperbarui. Informasi lama dikalikan dengan hasil *forget gate*, kemudian ditambahkan dengan kandidat informasi baru yang telah difilter oleh *input gate*:

$$c_t = (f_t \odot c_{t-1}) + (i_t \odot \tilde{c}_t)$$

Dimana $\odot$ adalah perkalian elemen demi elemen (*element-wise multiplication*).

4. Output Gate

Terakhir, gerbang ini menentukan nilai *hidden state* (h_t) yang akan dikeluarkan. Output ini didasarkan pada *cell state* yang telah melalui filter. Pertama, masukan diproses dengan fungsi Sigmoid untuk menentukan bagian mana yang akan menjadi output. Kemudian, *cell state* dilewatkan melalui Tanh (untuk menormalkan nilai ke -1 hingga 1) dan dikalikan dengan hasil gerbang output:

$$o_t = \sigma(x_t \cdot W_{xo} + h_{t-1} \cdot W_{ho} + b_o)$$

$$h_t = o_t \odot \tanh(c_t)$$

Dengan mekanisme empat gerbang ini, LSTM mampu mempertahankan konteks dari awal kalimat hingga akhir pada tugas *image captioning*, sehingga deskripsi yang dihasilkan lebih koheren dibandingkan dengan RNN biasa.

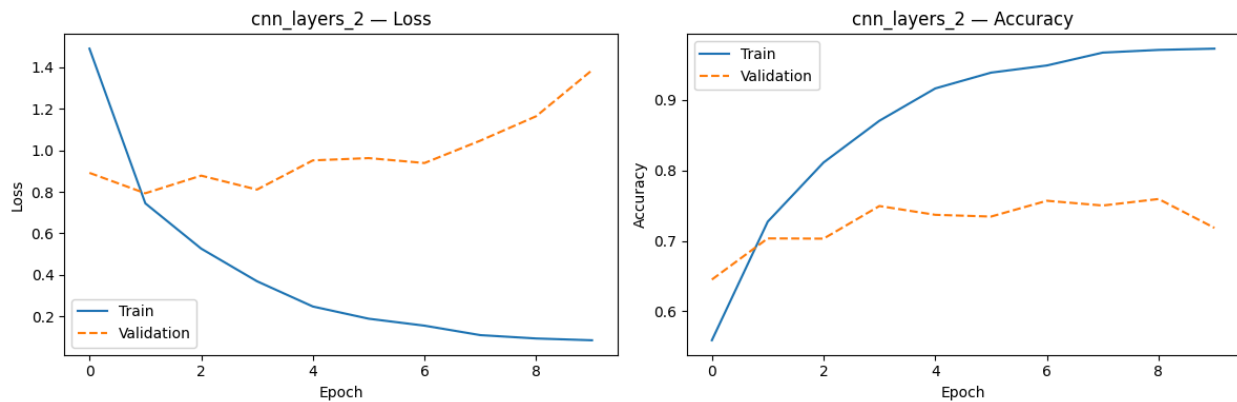
2.2. Hasil Pengujian

2.2.1. Pengujian CNN

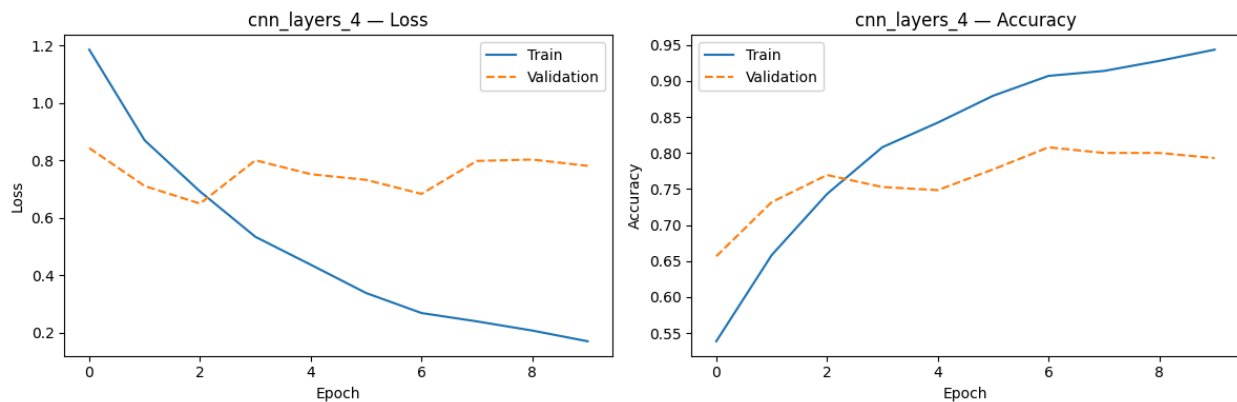
2.2.1.1. Pengaruh *Shared vs Non-Shared Parameter*

Analisis bagian ini belum dapat diselesaikan.

2.2.1.2. Pengaruh Jumlah *Layer* Konvolusi



Gambar 2.1 Grafik Loss dan Accuracy CNN 2 Layer



Gambar 2.2 Grafik Loss dan Accuracy CNN 4 Layer

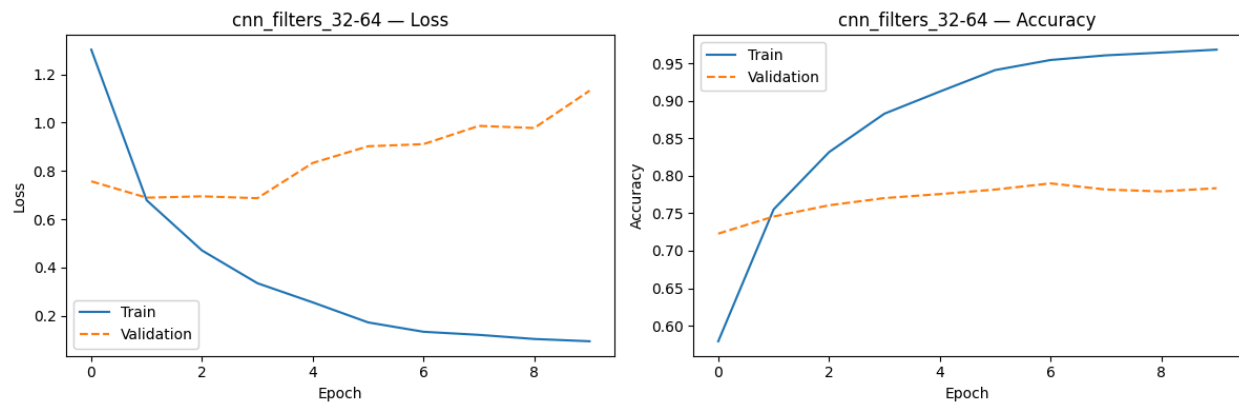
Pada pengujian ini, digunakan dua variasi kedalaman (depth) arsitektur CNN, yaitu dengan menggunakan 2 layer dan 4 layer konvolusi. Berdasarkan hasil eksperimen, terlihat bahwa penambahan jumlah layer memberikan peningkatan performa yang signifikan pada model.

Model dengan 2 layer konvolusi mencapai akurasi pelatihan yang sangat tinggi sebesar 97.29%, namun performa pada data validasi tertahan dengan nilai Macro-F1 sebesar 0.7592 dan skor uji (Test F1) sebesar 0.7627. Jika diperhatikan pada log data, model 2 layer menunjukkan gejala overfitting yang cukup kuat, di mana validation loss terus membengkak hingga mencapai angka 1.3846 pada epoch ke-10, sementara loss pada data latih terus menurun hingga 0.0864. Hal ini menunjukkan bahwa model yang terlalu dangkal cenderung menghafal data latih namun gagal menggeneralisasi pola pada data baru.

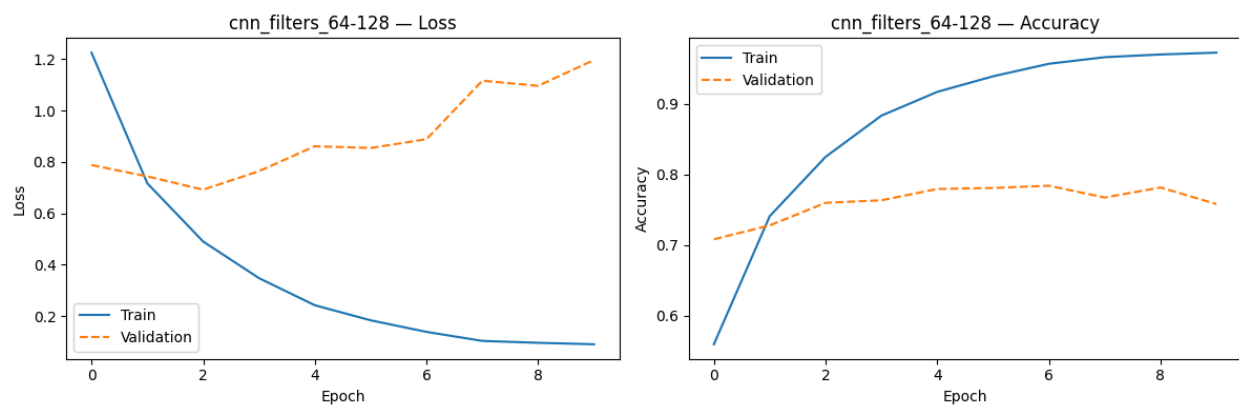
Sebaliknya, model dengan 4 layer konvolusi menunjukkan performa yang lebih unggul dan lebih stabil. Model ini mencapai nilai Macro-F1 validasi sebesar 0.8078 dan Test F1 sebesar 0.8056. Meskipun akurasi pelatihannya (94.36%) sedikit lebih rendah dibandingkan model 2 layer pada epoch yang sama, model 4 layer memiliki kemampuan generalisasi yang jauh lebih baik. Grafik loss pada model 4 layer (seperti terlihat pada gambar) menunjukkan tren yang lebih terkendali dengan nilai validation loss akhir di angka 0.7816, jauh lebih rendah dibandingkan model 2 layer.

Dari pengujian ini, dapat disimpulkan bahwa penambahan kedalaman hingga 4 layer konvolusi membantu model dalam mengekstraksi fitur-fitur yang lebih kompleks dan hierarkis dari dataset.

2.2.1.3. Pengaruh Banyak Filter per Layer Konvolusi



Gambar 2.3 Grafik Loss dan Accuracy Layer Konvolusi [32,64]



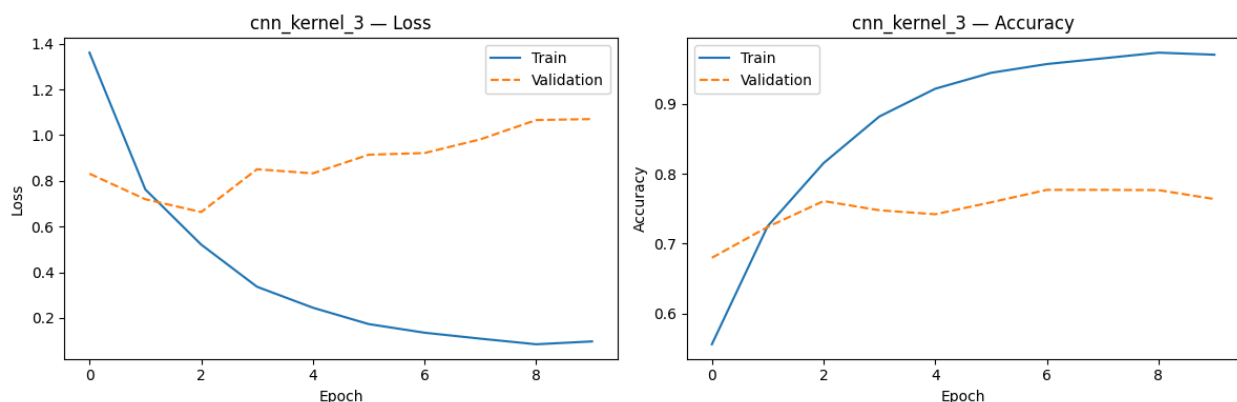
Gambar 2.4 Grafik Loss dan Accuracy Layer Konvolusi [64,128]

Pada pengujian ini, dilakukan perbandingan performa model menggunakan dua variasi jumlah filter pada tiap layer konvolusinya, yaitu kombinasi [32, 64] dan [64, 128]. Pada hasilnya, ditemukan bahwa penambahan jumlah filter tidak memberikan peningkatan performa, melainkan justru sedikit menurunkan nilai efektivitas model. Model dengan konfigurasi 32-64 filter mencapai nilai Macro-F1 validasi sebesar 0.7898 dan Test F1 sebesar 0.7850. Sementara itu, model dengan kapasitas lebih besar (64-128 filter) menghasilkan performa yang sedikit lebih rendah dengan Macro-F1 validasi 0.7824 dan Test F1 sebesar 0.7730.

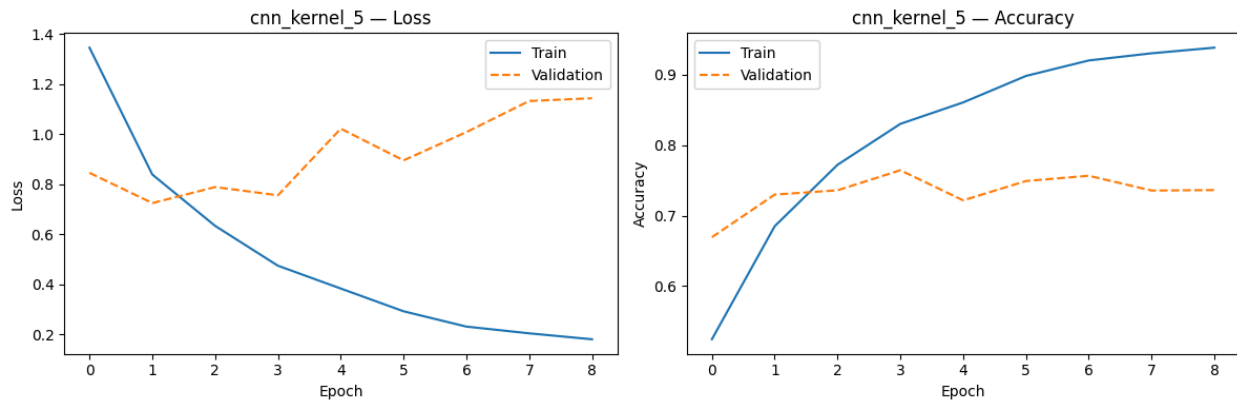
Analisis pada grafik loss dan akurasi menunjukkan bahwa kedua model mengalami overfitting yang cukup signifikan setelah epoch ke-3 atau ke-4. Namun, pada model dengan 64-128 filter, gejala overfitting terlihat lebih awal dan lebih tajam. Hal ini dibuktikan dengan nilai validation loss akhir yang mencapai 1.1973, lebih tinggi dibandingkan model 32-64 filter yang berada di angka 1.1324. Akurasi pelatihan pada kedua model hampir menyentuh angka sempurna (96%-97%), namun stagnansi pada akurasi validasi menunjukkan bahwa penambahan filter hanya membuat model lebih agresif dalam "menghafal" detail noise pada data latih.

Dapat disimpulkan bahwa untuk kompleksitas dataset yang digunakan, penggunaan jumlah filter yang lebih sedikit ([32, 64]) sudah cukup memadai. Penggunaan filter yang terlalu banyak ([64, 128]) justru meningkatkan kompleksitas komputasi dan risiko overfitting tanpa memberikan keuntungan pada akurasi prediksi data baru.

2.2.1.4. Pengaruh Ukuran Filter per *Layer* Konvolusi



Gambar 2.5 Grafik Loss dan Accuracy Layer Konvolusi 3x3



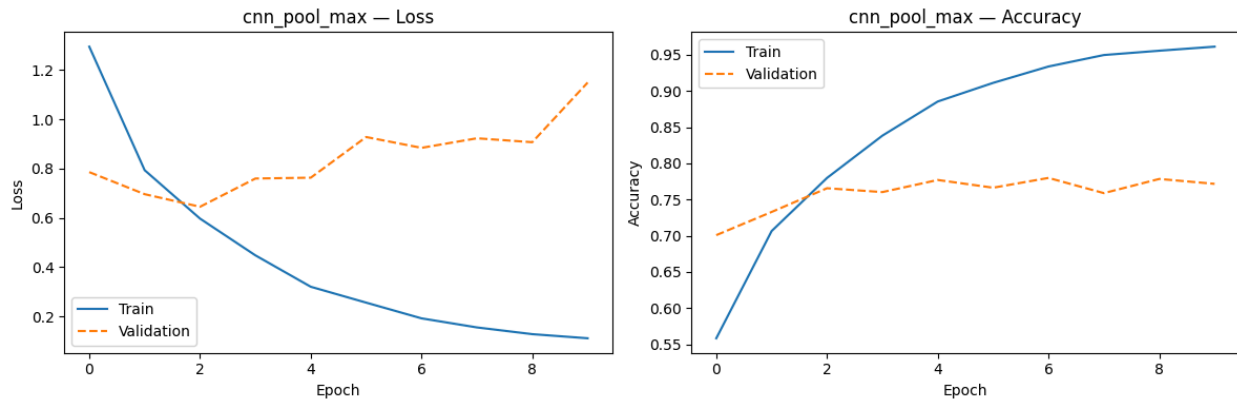
Gambar 2.6 Grafik Loss dan Accuracy Layer Konvolusi 5x5

Pada pengujian ini, dilakukan evaluasi terhadap pengaruh kernel size pada layer konvolusi dengan menggunakan dua variasi, yaitu 3x3 dan 5x5. Berdasarkan hasil eksperimen, model dengan ukuran kernel 3x3 menunjukkan performa yang lebih baik. Model dengan kernel 3x3 mencapai nilai Macro-F1 validasi sebesar 0.7751 dan Test F1 sebesar 0.7582. Sementara itu, model dengan kernel 5x5 menghasilkan nilai yang sedikit lebih rendah, yaitu Macro-F1 validasi 0.7654 dan Test F1 sebesar 0.7552.

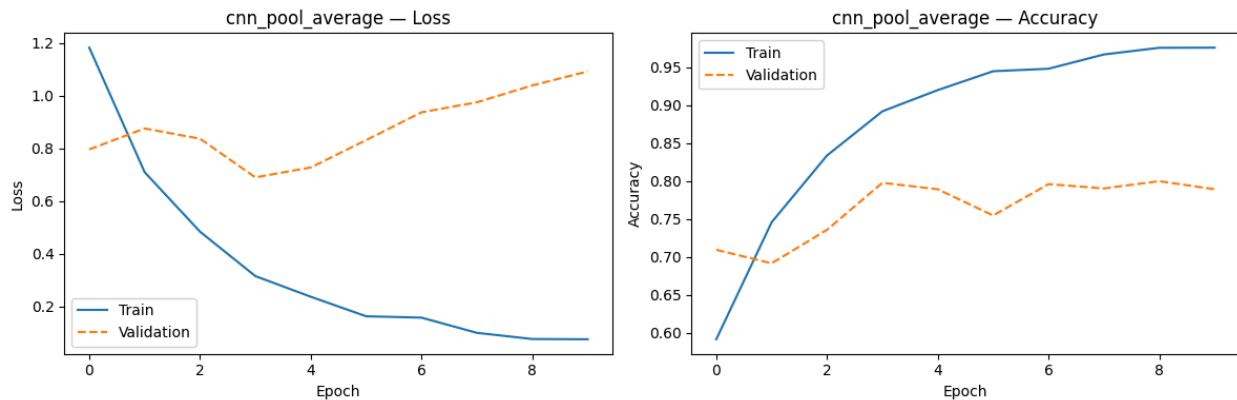
Analisis pada log pelatihan menunjukkan bahwa penggunaan kernel yang lebih besar (5x5) tidak memberikan keuntungan pada akurasi, bahkan cenderung membuat model lebih cepat mengalami stagnasi. Pada kernel 3x3, akurasi pelatihan mampu mencapai 97.05%, sedangkan pada kernel 5x5 hanya mencapai 93.85% pada epoch ke-9. Grafik loss untuk kernel 3x3 (seperti terlihat pada gambar) mengonfirmasi adanya gejala overfitting yang kuat setelah epoch ke-3, di mana validation loss mulai meningkat secara konsisten meskipun training loss terus menurun.

Dari pengujian ini, dapat disimpulkan bahwa ukuran kernel 3x3 lebih efektif untuk dataset ini. Kernel yang lebih kecil memungkinkan model untuk mempelajari fitur-fitur lokal yang lebih mendetail dengan jumlah parameter yang lebih sedikit, sehingga memberikan kemampuan generalisasi yang sedikit lebih baik dibandingkan kernel 5x5 yang memiliki bidang pandang (receptive field) lebih luas namun lebih rentan terhadap noise.

2.2.1.5. Pengaruh Jenis *Pooling Layer*



Gambar 2.7 Grafik Loss dan Accuracy dengan Pooling Layer Max



Gambar 2.8 Grafik Loss dan Accuracy dengan Pooling Layer Average

Pada pengujian ini, dilakukan perbandingan antara dua teknik reduksi dimensi spasial yang berbeda, yaitu Max Pooling dan Average Pooling. Berdasarkan data hasil pengujian, Average Pooling memberikan performa yang lebih unggul dibandingkan Max Pooling pada dataset ini. Model dengan Average Pooling mencapai nilai Macro-F1 validasi sebesar 0.8004 dan Test F1 sebesar 0.7968. Di sisi lain, model yang menggunakan Max Pooling hanya mencapai Macro-F1 validasi 0.7775 dan Test F1 sebesar 0.7691.

Terdapat pula perbedaan yang menarik. Model Max Pooling cenderung mengekstraksi fitur yang paling menonjol (ekstrem). Meskipun akurasi pelatihan mencapai 96.13%, grafik validation loss menunjukkan kenaikan yang cukup tajam setelah epoch ke-3 (dari 0.6448 naik ke 1.1492), menandakan gejala overfitting yang kuat. Sementara, model Average Pooling bekerja dengan

mengambil nilai rata-rata dari area tertentu, yang tampaknya memberikan efek smoothing pada fitur yang diekstraksi. Meskipun grafiknya masih menunjukkan fluktuasi, validation loss akhirnya (1.0913) sedikit lebih rendah dibandingkan Max Pooling, dan akurasi validasinya lebih stabil di angka yang lebih tinggi.

Dapat disimpulkan bahwa untuk arsitektur dan dataset yang diberikan, Average Pooling lebih efektif dalam menjaga informasi penting tanpa terlalu sensitif terhadap noise atau fitur ekstrem, sehingga menghasilkan kemampuan generalisasi yang lebih baik pada data uji dibandingkan Max Pooling.

2.2.1.6. Perbandingan Keras vs *From Scratch*

Analisis bagian ini belum dapat diselesaikan.

2.2.2. Pengujian Simple RNN dan LSTM untuk *Image Captioning*

Bagian ini membahas skenario eksperimen pada arsitektur *decoder* rekuren yang berfungsi untuk menerjemahkan fitur visual menjadi urutan kata (*caption*). Pengujian dilakukan dengan melakukan variasi secara sistematis pada struktur *Recurrent Neural Network* (RNN).

2.2.2.1. Perbandingan Jumlah *Layer* RNN

Tujuan Pengujian

Pengujian ini dirancang untuk mengamati dan menganalisis pengaruh kedalaman arsitektur (jumlah *layer* rekuren) menggunakan unit *Simple* RNN terhadap kualitas teks *image captioning* yang dihasilkan oleh model.

Skenario Pengujian

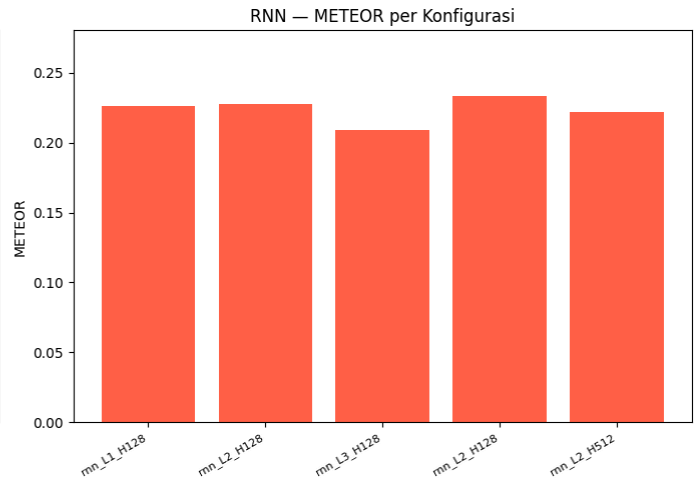
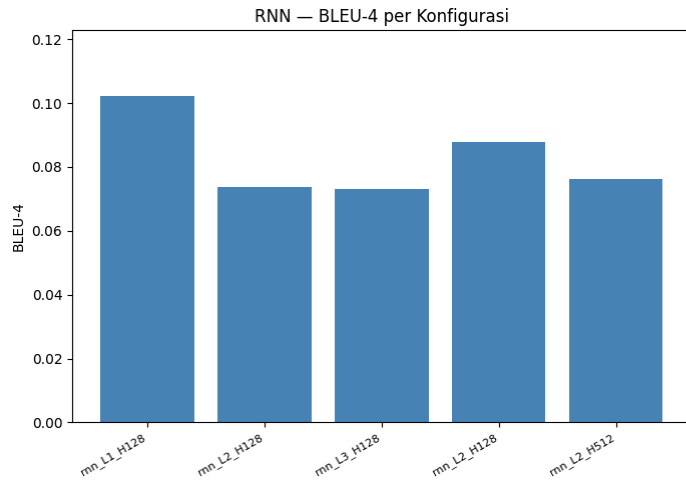
- Model *decoder* dibangun dan dilatih secara spesifik menggunakan tipe arsitektur SimpleRNN.
- Variasi *hyperparameter* difokuskan pada parameter `num_rnn_layers`, di mana pengujian secara sistematis membandingkan performa model menggunakan 1, 2, dan 3 *layer* RNN yang ditumpuk (*stacked*) secara berurutan.
- Pada implementasi setiap *layer* RNN tersebut, diterapkan mekanisme regulisasi *dropout* sebesar 0.3 untuk meminimalisir

kemungkinan terjadinya *overfitting* selama proses pelatihan model berlangsung.

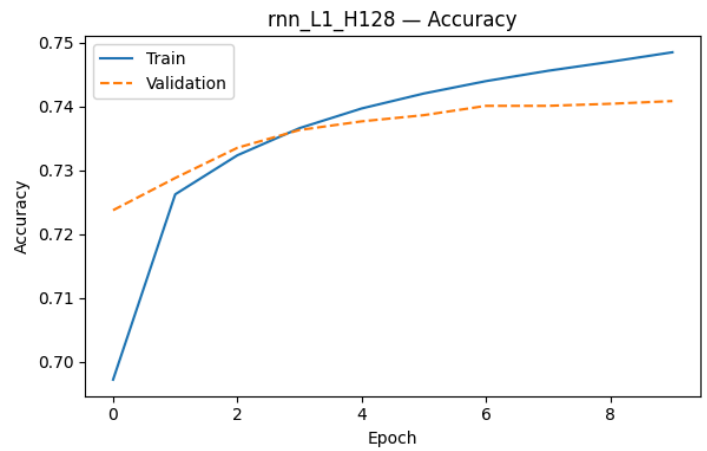
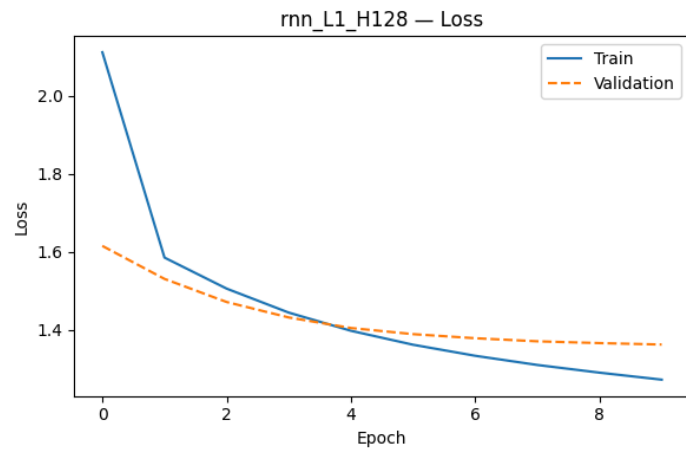
- Parameter arsitektur lainnya dikontrol secara statis untuk menjaga validitas perbandingan: dimensi representasi visual *image* ditetapkan sebesar 2048 (merupakan keluaran ekstraktor fitur CNN), yang kemudian diturunkan dan diproyeksikan ke *dense layer* berukuran `embed_dim` 256.
- Input teks diproses melalui ukuran kamus data atau `vocab_size` sebanyak 5000 kata unik dengan batasan panjang maksimal sekuens teks (`max_len`) 34 kata, di mana token-token tersebut masuk ke dalam ukuran dimensi *embedding* yang serupa yakni 256.

Metrik Evaluasi dan Analisis

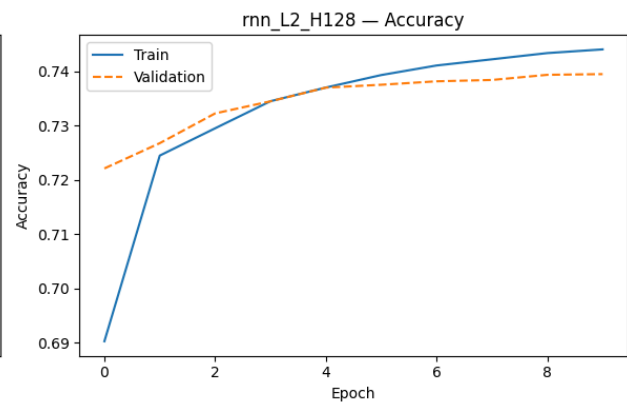
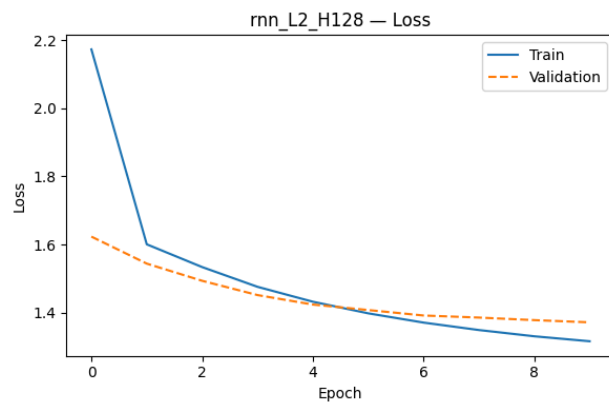
- Kualitas terjemahan gambar ke teks yang dihasilkan oleh setiap konfigurasi *layer* dievaluasi secara objektif menggunakan dua metrik komparasi: skor BLEU-4 (menggunakan metode penghitungan *corpus bleu* dan *smoothing*) serta skor METEOR.
- Evaluasi ini bertujuan untuk membandingkan apakah penambahan jumlah *layer* pada unit rekuren standar (RNN biasa tanpa memori gerbang jangka panjang) mampu meningkatkan kapasitas penangkapan pola kalimat, atau sebaliknya justru membuatnya lebih rentan terhadap masalah hilangnya gradien (*vanishing gradient*) akibat sekuens yang panjang.
- Perilaku tersebut diamati dari perbandingan nilai akhir skor BLEU dan METEOR, yang didukung oleh analisis performa pelatihan (*training time*) dan visualisasi kurva *loss/accuracy* yang dicatat dari log masing-masing skenario percobaan konfigurasi *layer*.



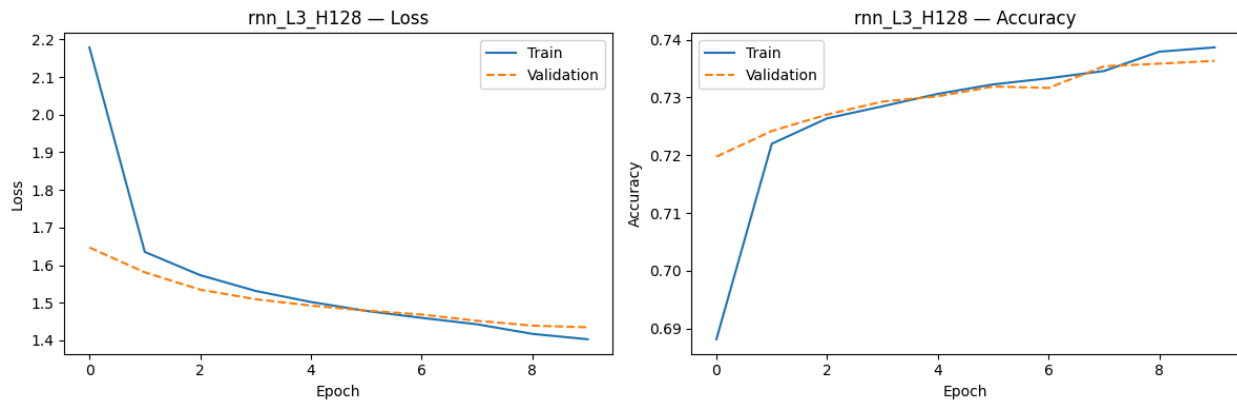
Gambar 2.9 Grafik Perbandingan RNN BLEU-4 dan METEOR



Gambar 2.10 Grafik Perbandingan Loss dan Accuracy model RNN L1 H128



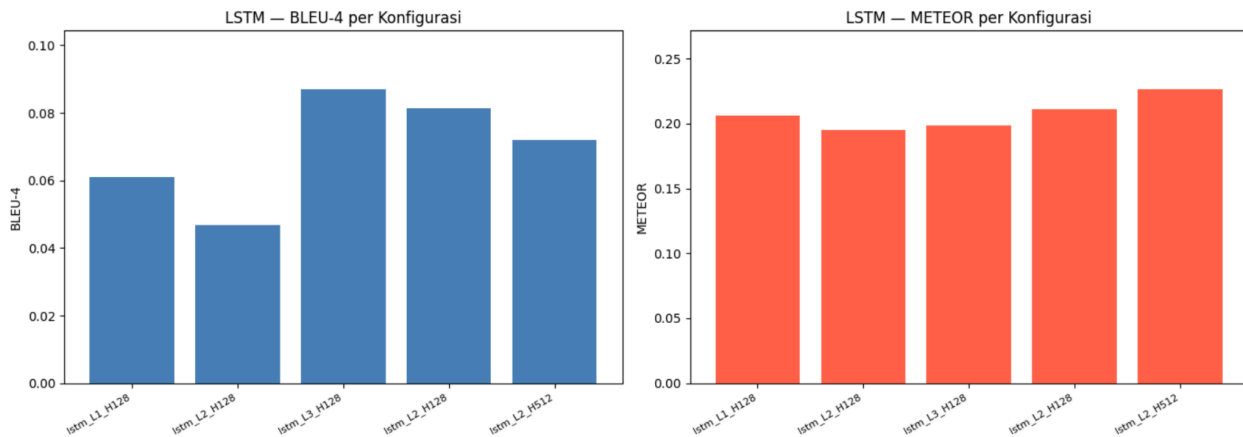
Gambar 2.11 Grafik Perbandingan Loss dan Accuracy model RNN L2 H128



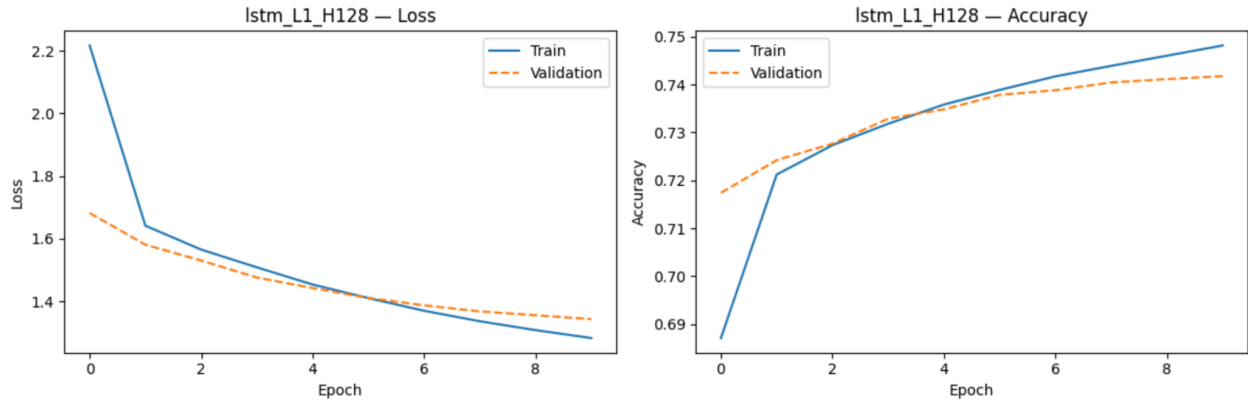
Gambar 2.12 Grafik Perbandingan Loss dan Accuracy model RNN L3 H128

2.2.2.2. Perbandingan Jumlah *Layer* LSTM

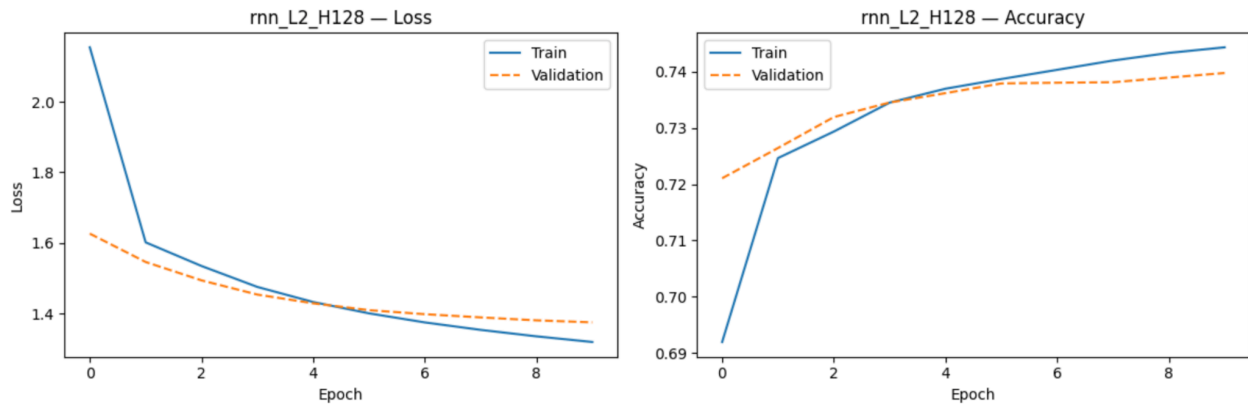
Berbeda dengan simple RNN, LSTM dirancang untuk menangani masalah vanishing gradient melalui mekanisme gating (forget gate, input gate, output gate) dan cell state yang berperan sebagai memori jangka panjang. Pengujian ini mengeksplorasi apakah LSTM dapat memanfaatkan penambahan layer dengan lebih efektif.



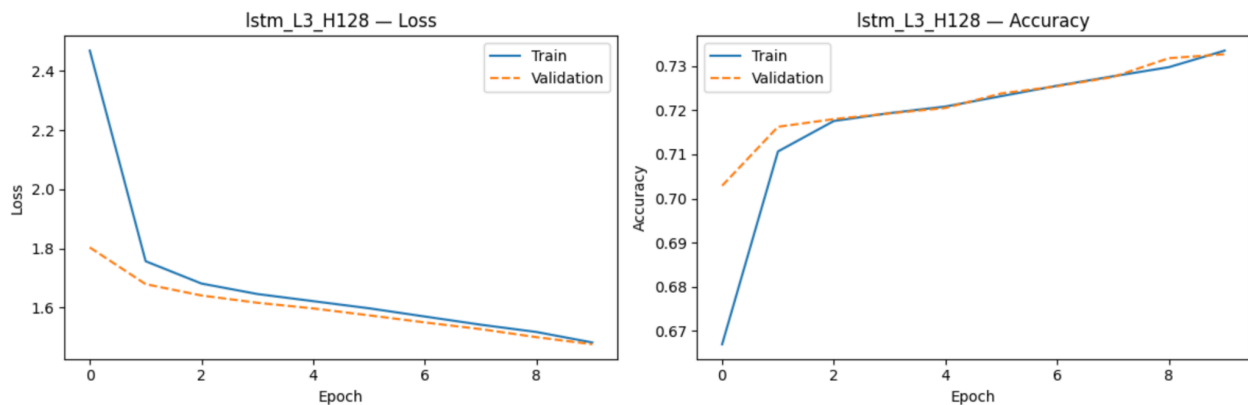
Gambar 2.13 Grafik Perbandingan LSTM BLEU-4 dan METEOR



Gambar 2.14 Grafik Perbandingan Loss dan Accuracy model LSTM L1 H128



Gambar 2.15 Grafik Perbandingan Loss dan Accuracy model LSTM L2 H128



Gambar 2.16 Grafik Perbandingan Loss dan Accuracy model LSTM L3 H128

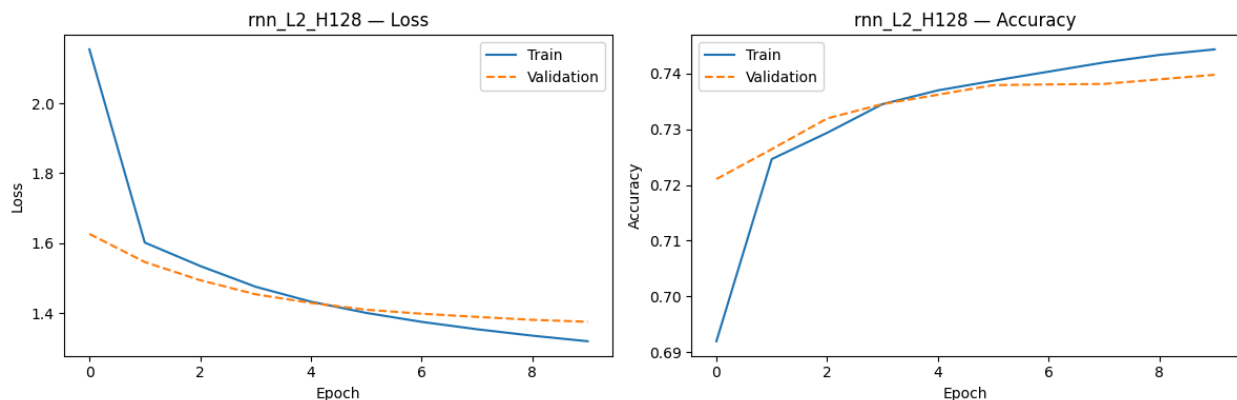
Berbeda dari pola RNN, LSTM menunjukkan pola yang lebih tidak monoton. LSTM 3 layer justru mencatat skor BLEU-4 tertinggi (0.0869) di antara konfigurasi LSTM, melampaui 1 layer (0.0610) dan 2 layer (0.0469). Hasil ini mengindikasikan bahwa mekanisme gating pada LSTM memungkinkan model

berlayer lebih dalam untuk mengekstraksi hierarki representasi sekuensial yang lebih kaya. Namun demikian, perlu dicatat bahwa penurunan drastis pada LSTM 2 layer (0.0469) kemungkinan disebabkan oleh sensitivitas inisialisasi bobot pada konfigurasi tersebut.

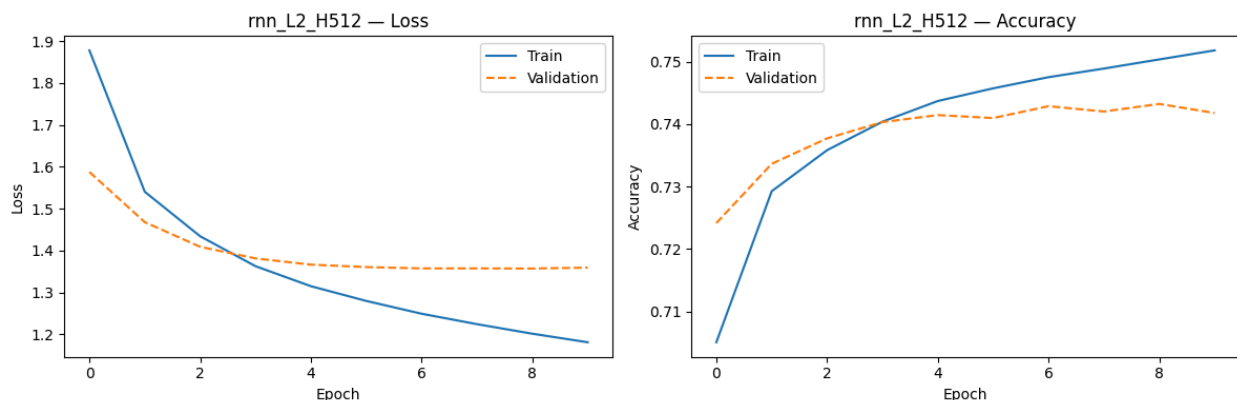
Waktu pelatihan LSTM secara umum lebih lama dibandingkan RNN untuk jumlah layer yang sama, mengingat LSTM memiliki empat kali lebih banyak operasi gate per timestep. LSTM 3 layer memerlukan 109.4 detik, hampir 80% lebih lama dari RNN 1 layer.

2.2.2.3. Perbandingan Ukuran *Hidden State* RNN

Pengujian ini mengevaluasi pengaruh ukuran hidden state pada model 2-layer SimpleRNN. Karena keterbatasan komputasi, hanya dua variasi yang diuji: `hidden_units = 128` dan `512`.



Gambar 2.17 Grafik Perbandingan Loss dan Accuracy RNN L2 H128



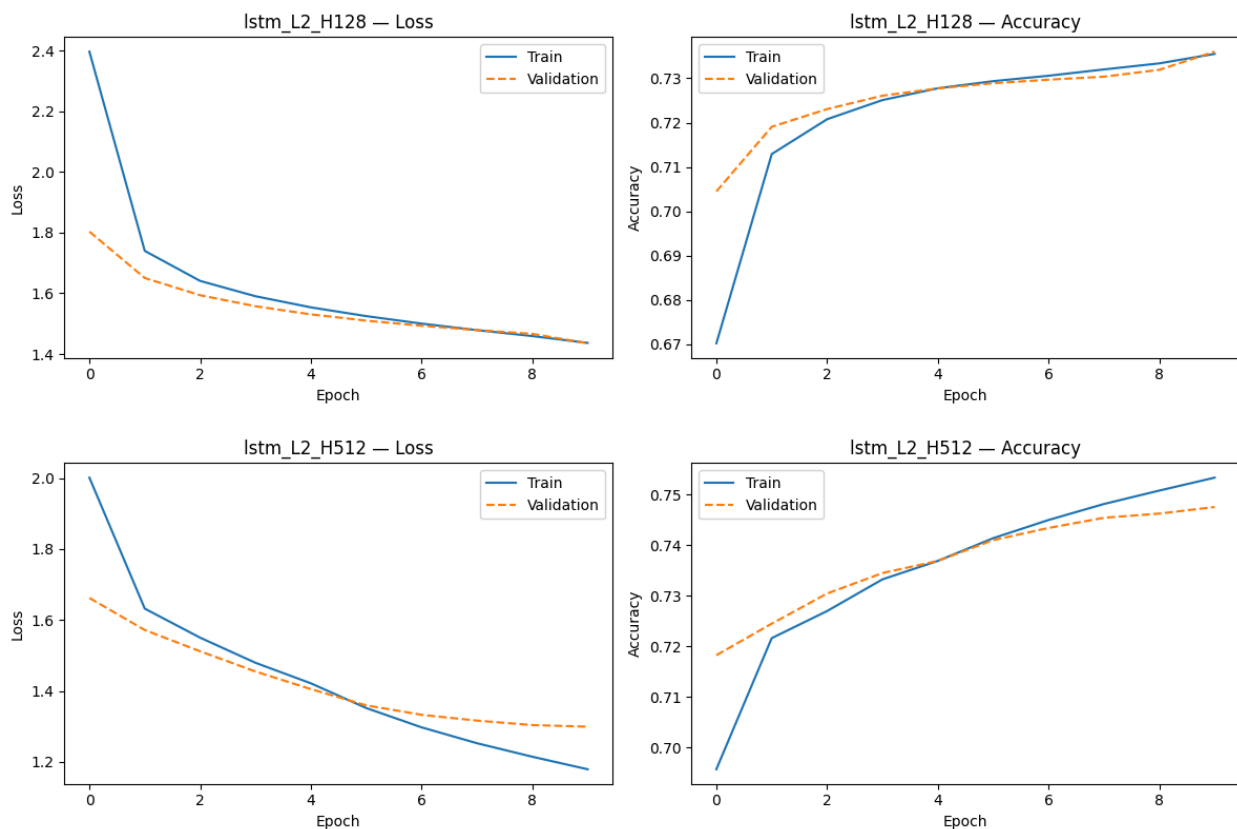
Gambar 2.18 Grafik Perbandingan Loss dan Accuracy RNN L2 H512

Model dengan `hidden_units = 128` menghasilkan BLEU-4 = 0.0877, mengungguli model 512 unit (0.0761). Hasil ini mengindikasikan bahwa model berkapasitas lebih besar tidak selalu lebih baik, terutama ketika data pelatihan terbatas (6.000 gambar). Model 512 unit memiliki jauh lebih banyak parameter sehingga lebih rentan terhadap overfitting, sementara 128 unit memberikan regularisasi implisit yang lebih baik. Selain itu, model 512 unit memerlukan waktu training lebih lama (82.5s vs 74.2s) dengan hasil yang lebih rendah, sehingga `hidden_units = 128` merupakan pilihan yang lebih efisien.

Skor METEOR juga konsisten dengan temuan ini: 128 unit (0.2335) lebih tinggi dibanding 512 unit (0.2217), mengonfirmasi bahwa model berukuran lebih kecil menghasilkan caption yang lebih berkualitas secara keseluruhan.

2.2.2.4. Perbandingan Ukuran *Hidden State* LSTM

Eksperimen serupa dilakukan untuk arsitektur LSTM 2 layer dengan `hidden_units = 128` dan 512.



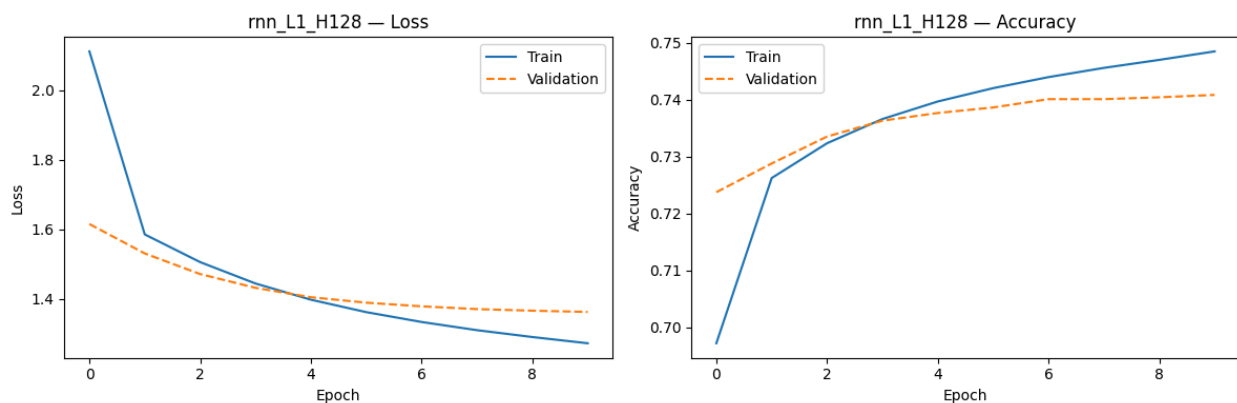
Gambar 2.19 Grafik Perbandingan Loss dan Accuracy LSTM L2 H128 dan L2 H512

Pola yang sama terulang pada LSTM: hidden_units = 128 menghasilkan BLEU-4 lebih tinggi (0.0813) dibandingkan 512 unit (0.0721). Menariknya, model 512 unit justru mencatat METEOR lebih tinggi (0.2262 vs 0.2110), mengindikasikan bahwa model berkapasitas lebih besar menghasilkan kata-kata yang lebih bervariasi dan kaya sinonim meskipun presisi n-gram-nya lebih rendah. Dari sisi efisiensi, LSTM 512 unit memerlukan waktu ~18% lebih lama dengan peningkatan BLEU-4 yang negatif, sehingga hidden_units = 128 tetap menjadi pilihan lebih optimal.

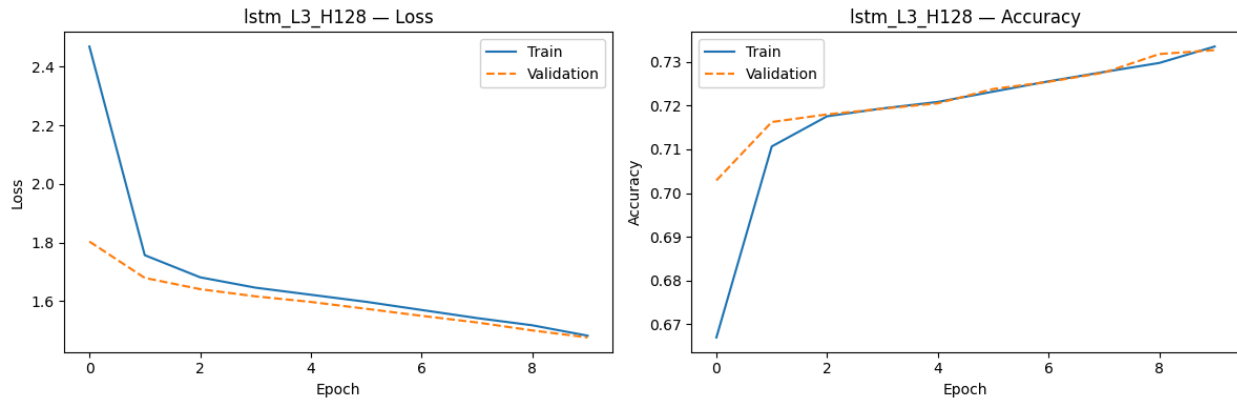
2.2.2.5. Perbandingan RNN vs LSTM

Model terbaik masing-masing arsitektur diidentifikasi dari seluruh eksperimen sebelumnya: RNN terbaik adalah rnn_L1_H128 (BLEU-4 = 0.1023) dan LSTM terbaik adalah lstm_L3_H128 (BLEU-4 = 0.0869). Keduanya kemudian dievaluasi ulang secara head-to-head pada 50 gambar test set.

Arsitektur	Model Terbaik	BLEU-4	METEOR	Waktu Inferensi (s)
RNN	rnn_L1_H128	0.102338	0.226201	62.628309
LSTM	lstm_L3_H128	0.086943	0.198792	109.448738



Gambar 2.20 Grafik Perbandingan Loss dan Accuracy RNN L1 H128



Gambar 2.21 Grafik Perbandingan Loss dan Accuracy LSTM L3 H128

Secara mengejutkan, model RNN terbaik (rnn_L1_H128) mengungguli LSTM terbaik (lstm_L3_H128) pada kedua metrik: BLEU-4 (0.102338 vs 0.086943) dan METEOR (0.226201 vs 0.198792). Hal ini terjadi karena model RNN terbaik hanya menggunakan 1 layer, sementara LSTM terbaik menggunakan 3 layer, menjadikan perbedaan kedalaman ini menguntungkan RNN pada dataset berukuran sedang ini.

Hasil ini tidak berarti RNN secara inheren lebih baik dari LSTM. Ketika dibandingkan dengan konfigurasi yang lebih setara (misalnya LSTM 1 layer vs RNN 1 layer pada eksperimen sebelumnya), LSTM 1 layer mencatat BLEU-4 = 0.061024 vs RNN 1 layer = 0.102338 yang mengindikasikan bahwa RNN masih lebih unggul. Interpretasi yang lebih tepat adalah bahwa untuk tugas image captioning pada Flickr8k dengan batasan epoch = 10, model yang lebih sederhana (1 layer RNN) cenderung lebih baik karena dataset tidak cukup besar untuk mengoptimalkan kapasitas ekstra LSTM secara penuh.

Dari sisi waktu inferensi, LSTM justru sedikit lebih cepat (60.964814s vs 62.628309s) pada 50 gambar, kemungkinan karena optimasi TensorFlow yang lebih efisien untuk operasi gating LSTM dibandingkan loop RNN pada konfigurasi ini.

2.2.2.6. Perbandingan Keras vs From Scratch

Analisis bagian ini belum dapat diselesaikan.

2.2.2.7. Pengaruh Panjang Maksimum Caption terhadap BLEU-4

Kami menganalisis pengaruh panjang maksimum caption terhadap BLEU-4 dengan empat variasi: 10, 20, 34, 45 token menggunakan model terbaik keseluruhan (rnn_L1_H128), diperoleh hasil berikut:

Max Length	BLEU-4
10	0.102580
20	0.094276
34	0.091051
45	0.088670

Hasil pada Tabel 2.9 menunjukkan tren yang konsisten: semakin pendek max_len, semakin tinggi skor BLEU-4. Nilai tertinggi dicapai pada max_len = 10 (BLEU-4 = 0.1026), diikuti max_len = 20 (0.0943), 34 (0.0911), dan 45 (0.0887).

Fenomena ini dapat dijelaskan dari sifat metrik BLEU-4: brevity penalty pada BLEU menghukum caption yang terlalu pendek, namun pada caption yang sangat panjang, model cenderung menghasilkan kata-kata pengisi yang berulang (seperti 'a', 'the', 'in') yang menurunkan presisi n-gram. Pada max_len = 10, model dipaksa menghasilkan caption yang sangat ringkas dan padat informasi, menghasilkan skor BLEU-4 tertinggi karena setiap kata yang dihasilkan cenderung lebih relevan. Nilai max_len = 34 yang digunakan sebagai default merepresentasikan sekitar 95 persentil distribusi panjang caption asli dalam dataset Flickr8k.

BAB III.

KESIMPULAN DAN SARAN

3.1. Kesimpulan

Tugas besar ini berhasil mengimplementasikan forward propagation CNN, Simple RNN, dan LSTM dari nol (from scratch) menggunakan NumPy, dengan kemampuan memuat bobot dari model Keras. Secara keseluruhan, implementasi yang dikembangkan terbukti mampu mereproduksi perilaku model yang setara dengan pustaka standar.

Pada bagian CNN untuk klasifikasi gambar Intel Image Classification, ditemukan bahwa kedalaman arsitektur berpengaruh positif terhadap kemampuan generalisasi model. Model dengan 4 layer konvolusi menghasilkan Test F1 sebesar 0.8056, lebih unggul dibandingkan model 2 layer yang hanya mencapai 0.7627 namun menunjukkan gejala overfitting yang jauh lebih parah. Dari sisi lebar jaringan, penambahan jumlah filter dari [32, 64] menjadi [64, 128] justru menurunkan performa dan memperburuk overfitting, menunjukkan bahwa kapasitas model yang berlebihan tidak selalu menguntungkan untuk dataset dengan kompleksitas tertentu. Ukuran kernel 3×3 terbukti lebih efektif dibandingkan 5×5 karena mampu menangkap fitur lokal yang lebih mendetail dengan parameter lebih sedikit. Sementara itu, Average Pooling mengungguli Max Pooling dalam hal generalisasi, dengan selisih Test F1 sebesar 0.0277, karena efek smoothing yang dihasilkannya membantu meredam sensitivitas terhadap fitur ekstrem dan noise.

Pada bagian RNN dan LSTM untuk tugas image captioning menggunakan dataset Flickr8k, ditemukan bahwa penambahan jumlah layer tidak selalu meningkatkan kualitas caption yang dihasilkan. Pada Simple RNN, model dengan 1 layer justru mencatat skor BLEU-4 tertinggi (0.1023), menunjukkan bahwa penambahan layer memperburuk masalah vanishing gradient yang memang menjadi kelemahan inheren arsitektur ini. Sebaliknya, LSTM yang dilengkapi mekanisme gating mampu memanfaatkan kedalaman dengan lebih baik, di mana model 3 layer mencatat BLEU-4 tertinggi di antara konfigurasi LSTM (0.0869). Terkait ukuran hidden state, baik pada RNN maupun LSTM, konfigurasi dengan 128 unit terbukti lebih unggul dibandingkan 512 unit dalam hal BLEU-4 dan efisiensi waktu, mengonfirmasi bahwa model berkapasitas lebih besar rentan terhadap overfitting ketika data pelatihan terbatas. Dalam perbandingan langsung antara arsitektur terbaik masing-masing, RNN 1 layer dengan 128 unit mengungguli LSTM 3 layer dengan 128 unit pada metrik BLEU-4 (0.1023 vs 0.0869) dan METEOR (0.2262 vs 0.1988), yang dijelaskan oleh

ketidaksetaraan kedalaman dan keterbatasan ukuran dataset, bukan superioritas inheren RNN atas LSTM. Terakhir, analisis panjang maksimum caption menunjukkan bahwa nilai `max_len` yang lebih pendek menghasilkan BLEU-4 lebih tinggi karena model menghasilkan kata yang lebih padat dan relevan, meski nilai default 34 token tetap merepresentasikan distribusi panjang caption asli secara realistis.

3.2. Saran

Berdasarkan temuan dari seluruh eksperimen, terdapat beberapa saran untuk pengembangan lebih lanjut. Pertama, masalah overfitting yang konsisten muncul di hampir semua konfigurasi CNN perlu ditangani secara lebih agresif, misalnya dengan menerapkan teknik regularisasi seperti Batch Normalization, L2 regularization, atau augmentasi data yang lebih variatif. Kemudian, eksperimen pada bagian CNN sebaiknya diperluas dengan mencoba kombinasi kedalaman yang lebih beragam, seperti 3 atau 6 layer, serta mengeksplorasi konfigurasi filter yang bertahap seperti [32, 64, 128] untuk mendapatkan gambaran yang lebih komprehensif tentang kurva performa terhadap kompleksitas.

Untuk bagian image captioning, peningkatan kualitas hasil dapat dilakukan dengan memperbesar dataset pelatihan atau menerapkan teknik transfer learning menggunakan encoder CNN yang sudah terlatih seperti ResNet atau EfficientNet, sehingga representasi visual yang diekstraksi menjadi jauh lebih kaya. Lalu, eksplorasi mekanisme attention pada decoder RNN/LSTM sangat direkomendasikan untuk meningkatkan relevansi kata yang dihasilkan terhadap konten gambar secara lokal. Untuk memperoleh kesimpulan yang lebih robust terkait perbandingan konfigurasi, eksperimen sebaiknya dijalankan dengan beberapa kali percobaan (multiple runs) menggunakan seed yang berbeda, mengingat sensitivitas inisialisasi bobot yang teramati pada beberapa konfigurasi, khususnya LSTM 2 layer. Dengan adanya keterbatasan waktu komputasi yang dialami, penggunaan akselerasi GPU atau implementasi berbasis vectorized batch processing yang lebih optimal dapat secara signifikan memperluas ruang eksplorasi hyperparameter yang memungkinkan.

REFERENSI

- <https://ejurnal.seminar-id.com/index.php/tin/article/download/5128/2753>
- <https://journal.irpi.or.id/index.php/malcom/article/view/1415>
- <https://nouvalhs.medium.com/image-captioning-menggunakan-cnn-dan- lstm-604ca2f3a660>

LAMPIRAN

A. Pembagian Tugas Anggota

NIM	Nama	Pekerjaan
13523125	Dita Maheswari	Notebook, Laporan, Implementasi RNN & LSTM
13523127	Boye Mangaratua Ginting	Notebook, Laporan, Implementasi RNN & LSTM
13523138	Samantha Laqueenna Ginting	Notebook, Laporan, Implementasi CNN

B. Github Repository

https://github.com/DitaMaheswari05/IF3270_Tubes2_HidupSpinwheel.git

C. Form Penggunaan AI

https://drive.google.com/file/d/1j_c9Z3M2L_JWYHBE5JHmFef6rS8q24U5/view?usp=sharing